Харківський національний університет радіоелектроніки
Факультет комп’ютерних наук
Катедра програмної інженерії

ЗВІТ

до лабораторної роботи №2

з дисципліни «Програмування мовою Dart»

на тему «Об’єктно-орієнтована модель предметної області»

Виконав

ст. гр. ПЗПІ-23-1

Левченко Єгор

Перевірив

асистент катедри ПІ

Михайло Дмитрович Баличев

Харків 2026

1 ОПИС РОБОТИ

1.1 Мета роботи

Отримання навичок проєктування об’єктно-орієнтованої моделі предметної області засобами мови `Dart`: класів і конструкторів, ієрархії типів, домішок, методів розширення, узагальнень, записів і зіставлення зі зразком.

1.2 Завдання

Переробити програму лабораторної роботи 1 (консольний каталог музичних треків) в об’єктно-орієнтовану модель: функції розбору й агрегації зберігаються, але працюють із об’єктами предметної області. Реалізувати:

- незмінний клас-сутність із геттерами, `toString()`, іменованим конструктором `fromCsv` і фабричними конструкторами;
- версію класу з первинним конструктором (Dart 3.13) та порівняння обсягу коду;
- перевизначені `==` і `hashCode` з перевіркою поведінки у `Set`;
- запечатану ієрархію з щонайменше трьома підтипами, поліморфний метод і `switch`-вираз без `default`;
- домішку з обмеженням `on`, підмішану до двох класів;
- метод розширення вбудованого типу та узагальнене сховище з обмеженням параметра типу;
- перелічення з членами, запис із деструктуризацією та зіставлення зі зразком з охоронцем `when`;
- перевірку проєкту командами `dart analyze`, `dart format` і `dart fix --dry-run`.

Предметна область та вхідні дані не змінилися: це каталог музичних треків і константний список `sampleTrackCsvRows` із 15 рядків, серед яких 4 пошкоджені.

2 ОПИС ВИКОНАНОЇ РОБОТИ

Весь вихідний код опубліковано на платформі GitHub [1] (гілка `lab-2`). Обґрунтування прийнятих рішень також наведено у файлі `README.md` репозиторію.

2.1 Структура програми

Модулі розбору, валідації та обробки з лабораторної роботи 1 перенесено до каталогу `lib/src/` відповідно до Effective Dart [3]. Єдиним публічним фасадом пакета є `lib/core.dart`, який реекспортує всі сутності для `bin/trad.dart` і тестів. Склад проєкту наведено в таблиці 2.1.

Таблиця 2.1 – Склад проєкту

Файл	**Призначення**
`lib/core.dart`	Публічний фасад; метрики каталогу, форматування звіту
`lib/src/csv_parsing.dart, processing.dart, validation.dart`	Розбір рядків CSV, обробка колекцій, валідація (перенесено з лабораторної роботи 1)
`lib/src/models/track.dart`	Незмінна сутність `Track`
`lib/src/models/audio_item.dart`	Запечатана ієрархія `AudioItem`
`lib/src/models/identifiable.dart, mixins.dart`	Інтерфейс `Identifiable`, домішка `Auditable`
`lib/src/models/genre.dart, album.dart`	Перелічення `MusicGenre`, клас `Album`
`lib/src/models/repository.dart`	Узагальнене сховище `Repository<T>`
`lib/src/models/extensions.dart`	Методи розширення
`test/trad_test.dart`	Модульні тести (21)

2.2 Клас-сутність Track

`Track` — незмінний клас: усі поля (`id`, `trackName`, `albumName`, `artists`, `releaseDate`, `duration`, `genre`) оголошено як `final`. Обчислювані властивості подано геттерами `releaseYear`, `isSingle`, `leadArtist` і `formattedDuration`; перевизначено `toString()`. Список авторів у конструкторах загортається в `List.unmodifiable`, щоб незмінність не порушувалась ззовні. Генеративний

іменований конструктор `Track.create` приймає іменовані параметри й делегує ініціалізацію основному конструктору.

Іменований конструктор `Track.fromCsv` замінює функцію розбору з лабораторної роботи 1. На відміну від неї, він не повертає `null`, а викидає `FormatException` з описом проблеми. Щоб конвеєр обробки, як і раніше, міг пропускати пошкоджені рядки, додано статичний метод `Track.tryFromCsv`, який перехоплює виняток і повертає `null`.

```
factory Track.fromCsv(String line) {
  final fields = parseCsvRow(line);
  if (fields.length != _expectedCsvColumnCount) {
    throw FormatException('Expected ...', line);
  }
  final releaseDate = DateTime.tryParse(fields[3]);
  final durationMs = int.tryParse(fields[4]);
  // якщо одне з них null – FormatException
  return Track(id, trackName, albumName, artists,
      releaseDate, Duration(milliseconds: durationMs));
}

static Track? tryFromCsv(String line) {
  try {
    return Track.fromCsv(line);
  } on FormatException {
    return null;
  }
}
```

Фабричний конструктор `Track.cached` реалізує кешування: екземпляри зберігаються у статичній мапі за `id`, тому повторний виклик з тим самим `id` повертає наявний об’єкт. Фабричний конструктор `Track.withDefault` повертає трек зі значеннями за замовчуванням (назва `Untitled Track`, альбом `Unknown Album`, тривалість 3 хв).

```
static final Map<String, Track> _cache = {};

factory Track.cached({required String id, ...}) {
  return _cache.putIfAbsent(
    id,
    () => Track(id, trackName, albumName,
        List.unmodifiable(artists), releaseDate,
        duration, genre: genre),
```

```
  );
}
```

2.3 Первинний конструктор Dart 3.13

Клас `Track` записано із застосуванням синтаксису первинного конструктора, у якому поля оголошуються безпосередньо в заголовку класу.

```
class Track(
  @override final String id,
  final String trackName,
  final String albumName,
  final List<String> artists,
  final DateTime releaseDate,
  final Duration duration, {
  final MusicGenre genre = MusicGenre.soundtrack,
}) extends Identifiable with Auditable {
  this;
  // ...
}
```

Порівняння з генеративним конструктором:

– обсяг коду: у традиційному класі кожне поле записується двічі – в оголошенні та в списку параметрів конструктора; первинний конструктор поєднує їх, що скорочує шаблонний код приблизно на 15–20 рядків;

– що не змінилося: поля залишаються типізованими й незмінними, доступ до них (`track.trackName`) однаковий, клас так само має іменовані та фабричні конструктори, геттери й перевизначені оператори;

– що змінилося: вторинні конструктори мусять делегувати первинному (`: this(...)`); первинний конструктор у поточній версії компілятора не позначається `const`.

2.4 Перевизначення == і hashCode

Структурне порівняння `Track` виконується за всіма значущими полями; для списку `artists` порівнюються елементи, а `hashCode` обчислюється через `Object.hash` та `Object.hashAll(artists)`.

```
@override
bool operator ==(Object other) {
```

```
  if (identical(this, other)) return true;
  if (other is! Track) return false;
  return id == other.id &&
      trackName == other.trackName &&
      _listEquals(artists, other.artists) &&
      /* albumName, releaseDate, duration, genre */;
}

@override
int get hashCode => Object.hash(id, trackName, albumName,
    Object.hashAll(artists), releaseDate, duration, genre);
```

Якщо перевизначено і оператор ==, і метод `hashCode`, розмір `Set<Track>` дорівнює одиниці. У цьому випадку об’єкти мають однаковий хеш-код і потрапляють в один кошик, де оператор == підтверджує дублікат.

Якщо ж реалізовано лише ==, розмір множини становить два. Хеш-коди об’єктів виявляються різними, тому вони потрапляють у різні кошики, а перевірка через == навіть не викликається.

2.5 Запечатана ієрархія AudioItem

Базовий клас `AudioItem` оголошено як `sealed`; він задає абстрактні члени `title`, `duration` та поліморфний метод `formatSummary()`. Підтипи: `StudioTrack`, `LiveRecording`, `RemixTrack` і `PodcastEpisode` (четвертий підтип додавався на кроці перевірки вичерпності). Кожен підтип реалізує `formatSummary()` по-своєму.

Функція `describeAudioItem` обчислює опис за підтипом через `switch`-вираз без `default` і водночас демонструє зіставлення зі зразком з охоронцем `when`. Охоронні гілки не враховуються під час перевірки вичерпності, тому для кожного підтипу є гілка без охоронця.

```
String describeAudioItem(AudioItem item) {
  return switch (item) {
    StudioTrack(:final duration, :final studioName)
        when duration.inMinutes >= 10 =>
      'Epic Studio Masterpiece ($studioName, ...)',
    StudioTrack(:final studioName) =>
      'Studio Production recorded at $studioName',
    // LiveRecording, RemixTrack, PodcastEpisode ...
```

```
  };
}
```

Якщо додати `PodcastEpisode` і не обробити його у виразі, статичний аналіз повідомляє про помилку: додавання нового підтипу не залишається непоміченим.

```
error - The type 'AudioItem' isn't exhaustively matched by
the switch cases since it doesn't match the pattern
'PodcastEpisode()'. Try adding a wildcard pattern or cases
that match 'PodcastEpisode()'.
- non_exhaustive_switch_expression
```

2.6 Домішка Auditable

Домішка `Auditable` додає сутностям формування стандартизованого URN (`urn`) та запису аудиту (`formatAuditEntry`). Обмеження `on Identifiable` звужує коло класів, до яких її можна підмішати, і дає доступ до `id` усередині домішки без приведення типів. Домішку підмішано до класів `Track` та `Album`.

```
mixin Auditable on Identifiable {
  String get urn => 'urn:trad:entity:$id';

  String formatAuditEntry({required String action}) =>
      '[$action] Entity(id: "$id")';
}

class Album extends Identifiable with Auditable { ... }
```

Успадкування тут не підходить з таких причин. По-перше, трек і альбом – різні сутності: спільний штучний предок зв’язав би їх жорсткою ієрархією без змістовного відношення «є». По-друге, Dart підтримує лише одинарне успадкування, тому клас, що успадкував поведінку від спільного предка, не зміг би успадковувати базовий клас власної гілки (наприклад, `AudioItem`). Домішка ж додає поведінку, не займаючи місця в ланцюжку успадкування.

2.7 Методи розширення

Оголошено розширення вбудованих типів, корисні для предметної області:

– `DurationFormatting` на `Duration`: `toMmSs()` (формат `m:ss` або `h:mm:ss`) і `toMinutesSecondsUk()` (формат `29 хв 25 с`);

– DateTimeDomainExtensions на DateTime: toIsoDateOnly() та ageInDays({targetDate});
– TrackIterableExtensions на Iterable<Track>: totalDuration, averageDuration і allUniqueArtists.

```
extension TrackIterableExtensions on Iterable<Track> {
  Duration get totalDuration => fold<Duration>(
      Duration.zero, (sum, t) => sum + t.duration);

  Set<String> get allUniqueArtists =>
      expand((t) => t.artists).toSet();
}
```

2.8 Узагальнене сховище

Клас Repository<T extends Identifiable> – типізоване сховище в пам'яті з операціями add, addAll, findById, where, getAll, remove і clear.

```
class Repository<T extends Identifiable> {
  final Map<String, T> _items = {};

  void add(T item) => _items[item.id] = item;
  T? findById(String id) => _items[id];
  Iterable<T> where(bool Function(T) test) =>
      _items.values.where(test);
}
```

Обмеження T extends Identifiable дає дві речі. Перше – типобезпеку: у сховище можна покласти лише об'єкти, які гарантовано мають id, а довільний тип спричинить помилку компіляції. Друге – прямий доступ до item.id як до ключа без dynamic або явного приведення типу.

2.9 Перелічення MusicGenre

Перелічення MusicGenre описує жанр треку: значення rock, pop, electronic, soundtrack, classical. Кожне значення має поля code, displayName та allowsExplicitLyrics, які ініціалізує константний конструктор. Методи formatBadge() (наприклад, [OST] Soundtrack) і matches(String query) (порівняння з кодом або назвою без урахування регістру) інкапсулюють поведінку жанру. Жанр використано в полі Track.genre.

2.10 Записи та зіставлення зі зразком

Функція `computeCatalogMetrics` повертає кілька значень одним іменованим записом `CatalogMetrics`; виклична сторона розбирає його деструктуризацією. Функція `categorizeTrackPlayback` класифікує трек за `switch`-виразом з охоронцями `when`.

```
typedef CatalogMetrics = ({Duration totalDuration,
  Duration averageDuration, int trackCount, Track? longestTrack});

final (:totalDuration, :averageDuration, :trackCount,
    :longestTrack) = computeCatalogMetrics(validTracks);

String categorizeTrackPlayback(Track track) =>
  switch (track) {
    Track(:final duration) when duration.inMinutes >= 10 =>
      'Epic (${duration.inMinutes} min)',
    // Interlude, Collaboration ...
    Track() => 'Standard Track (${track.formattedDuration})',
  };
```

2.11 Оператор !, ключове слово late, тип dynamic і залежності

У коді проєкту не вжито жодного разу:

- оператор `!`: можливі значення `null` обробляються явними перевірками (`== null`), відсіюванням через `whereType<Track>()` та оператором `??`;
- ключове слово `late`: усі змінні ініціалізуються в місці оголошення або через параметри;
- тип `dynamic`: усі типи задано явно або виведено аналізатором; правила `no_dynamic_casts` і `no_raw_types` увімкнено в `analysis_options.yaml`.

Сторонні залежності з `pub.dev`: `path` – офіційний пакет для кросплатформної роботи зі шляхами; `lints` – набір правил `recommended` для статичного аналізу; `test` – офіційний засіб модульного тестування. Усі три пакети є офіційними пакетами екосистеми Dart, тому альтернативи не розглядалися.

3 РЕЗУЛЬТАТИ ВИКОНАННЯ

Аналізатор не має зауважень, усі 21 тест пройдено, форматування не змінило жодного файлу, `dart fix` не запропонував виправлень. Результати запуску інструментів перевірки повністю збігаються з наведеними в `README.md`.

```
> dart analyze
Analyzing trad...
No issues found!

> dart test
00:00 +21: All tests passed!

> dart format --output=none .
Formatted 14 files (0 changed) in 0.07 seconds.

> dart fix --dry-run
Computing fixes in trad (dry run)...
Nothing to fix!
```

Фрагмент виводу демонстраційної програми `bin/trad.dart` для дедуплікації у `Set` та поліморфних методів наведено нижче.

```
trackCopy1 == trackCopy2: true
Set size with two equal elements: 1 (expected: 1)
- Polymorphic summary: Podcast episode #1: "History of
  Synthesizers" (45m)
```

4 ВИСНОВКИ

У даній лабораторній роботі програму лабораторної роботи 1 перероблено в об’єктно-орієнтовану модель предметної області засобами мови Dart. Побудовано незмінний клас `Track` з іменованим і фабричними конструкторами, перевизначеними `==` та `hashCode`; запечатану ієрархію `AudioItem`, вичерпність `switch` якої перевіряє компілятор; домішку `Auditable` з обмеженням `on`; методи розширення; узагальнене сховище; перелічення з членами, запис і зіставлення зі зразком з охоронцем. Первинний конструктор Dart 3.13 скоротив обсяг коду без зміни поведінки класу. Код проходить `dart analyze` без зауважень, усі тести пройдено.

5 ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Левченко Є. dart-lab-trad (branch: lab-2) : репозиторій вихідного коду. GitHub. URL: https://github.com/NureLevchenkoYehor/dart-lab-trad/tree/lab-2 (дата звернення: 10.11.2026).
2. The Dart language. Dart documentation. URL: https://dart.dev/language (дата звернення: 10.11.2026).
3. Effective Dart. Dart documentation. URL: https://dart.dev/effective-dart (дата звернення: 10.11.2026).